

Wstępny zarys projektu

Układ regulacji automatycznej temperatury z
interfejsem użytkownika oparty o platformę STM32



POLITECHNIKA POZNAŃSKA

Wydział Automatyki, Robotyki i Elektrotechniki

Michał Błotniak

159409

10 grudzień 2025

Spis treści

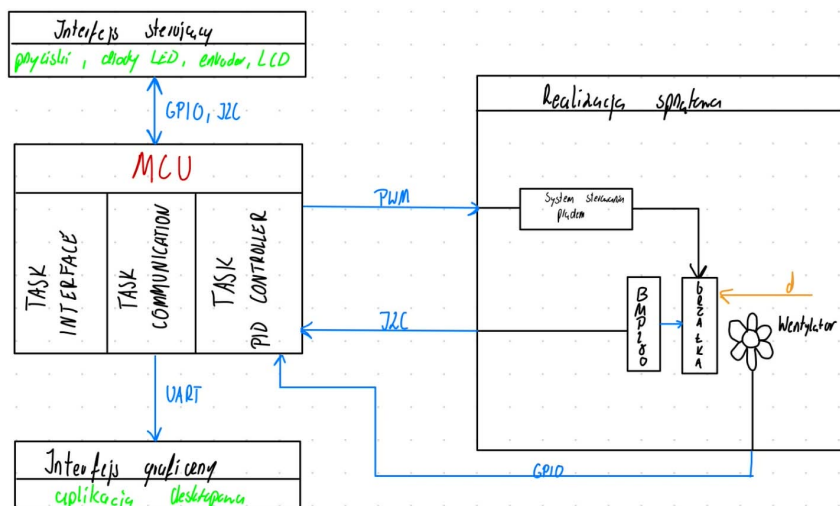
1	Szczegółowa koncepcja i architektura systemu	2
1.1	Charakterystyka obiektu regulacji	2
2	Architektura oprogramowania (Firmware)	4
2.1	Organizacja projektu i środowisko programistyczne	4
2.1.1	Struktura plików i katalogów	4
2.1.2	System budowania (CMake) i Dokumentacja (Doxygen)	5
2.2	System operacyjny czasu rzeczywistego (FreeRTOS)	5
2.3	Diagram przepływu danych i synchronizacji	5
2.4	Szczegółowy opis zadań systemowych (Tasks)	6
2.5	Model danych i współdzielenie zasobów	6
2.5.1	Zadanie sterowania (PID_Task)	7
2.5.2	Zadanie interfejsu użytkownika (Interface_Task)	8
2.5.3	Zadanie komunikacji (Communication_Task)	8
2.5.4	Odbiór komend (Interrupt-Based RX)	8
2.5.5	Nadawanie telemetry (Cyclic TX)	9
3	Aplikacja monitorująca i wizualizacyjna (PC)	10
3.1	Architektura wielowątkowa aplikacji	10
3.2	Protokół komunikacyjny i parsowanie danych	11
3.3	Funkcjonalności interfejsu	11
4	Realizacja sprzętowa urządzenia	13
4.1	Jednostka centralna (MCU Section)	13
4.2	Tor pomiarowy i interfejs użytkownika (Sensor & HMI)	13
4.2.1	Sekcja wyświetlacza (LCD Section)	14
4.2.2	Sekcja enkodera (Interface Section)	14
4.2.3	Sekcja czujnika (BMP280 Section)	14
4.3	Elementy wykonawcze (Actuators)	14
4.3.1	Sekcja grzewcza (Heating Section)	15
4.3.2	Sekcja chłodzenia (Fan Section)	15
5	Identyfikacja parametryczna obiektu i synteza układu regulacji	16
5.1	Metodyka eksperymentu czynnego	16
5.2	Wybór struktury modelu i estymacja parametrów	16
5.3	Weryfikacja jakości modelu i analiza błędów	17
5.4	Synteza regulatora PID	18
5.5	Analityczny dobór nastaw metodą lokowania biegunów	19
5.5.1	Wyprowadzenie zależności	19
5.5.2	Obliczenia dla zidentyfikowanego modelu	20
5.5.3	Wynik doboru nastaw	20

1 Szczegółowa koncepcja i architektura systemu

Projekt dotyczy realizacji mikroprocesorowego układu sterowania temperaturą w pętli zamkniętej. System został zaprojektowany jako układ typu SISO (Single Input, Single Output), gdzie wielkością sterującą jest moc dostarczana do elementu grzejnego, a wielkością regulowaną temperatura mierzona przez cyfrowy czujnik BMP280.

Całość rozwiązania opiera się na 32-bitowym mikrokontrolerze z rodziny STM32, pracującym pod kontrolą systemu czasu rzeczywistego FreeRTOS. Wybór architektury opartej na RTOS podyktowany był koniecznością zapewnienia determinizmu czasowego dla algorytmu regulacji przy jednoczesnej obsłudze asynchronicznych zdarzeń pochodzących od interfejsu użytkownika.

Ogólny schemat blokowy przedstawiono na Rysunku 1.



Rysunek 1: Schemat blokowy sprzętowo-programowej struktury systemu

1.1 Charakterystyka obiektu regulacji

Obiektem sterowania jest model termiczny ("Realizacja sprzętowa"), który charakteryzuje się inercyjnością oraz występowaniem opóźnienia transportowego. W skład modelu sprzętowego wchodzi:

- **Aktuator (Grzałka - rezystor):** Element wykonawczy sterowany sygnałem PWM (Pulse Width Modulation). Zmiana współczynnika wypełnienia sygnału PWM przez mikrokontroler pozwala na płynną regulację średniej mocy wydzielanej na rezystorze, co stanowi sygnał

sterujący $u(t)$. Dodatkowo, gdy temperatura zadana jest niższa niż obecnie mierzona na grzałce, uruchamiane jest dodatkowe urządzenie wykonawcze - wentylator, który zapewnia obniżenie temperatury do zadanego poziomu.

- **Zakłócenie (d):** Zakłóceniem jakie może wystąpić w układzie jest na przykład, wymuszenie wzrostu temperatury poprzez ściśnięcie palcem rezystora/czujnika, przyłożenie płomienia zapalniczki lub też w drugą stronę, podmuch chłodniejszego powietrza.
- **Sensor (BMP280):** Pomiar temperatury realizowany jest przez cyfrowy czujnik BMP280, komunikujący się z jednostką centralną za pośrednictwem magistrali I2C. Wybór czujnika cyfrowego eliminuje konieczność stosowania zewnętrznych układów kondycjonowania sygnału analogowego, jednak wprowadza konieczność obsługi protokołu komunikacyjnego w ściśle określonych reżimach czasowych.

2 Architektura oprogramowania (Firmware)

Oprogramowanie wbudowane dla mikrokontrolera STM32L476RG stanowi kluczowy element projektu, integrujący warstwę sprzętową z algorytmami sterowania. Projekt został zrealizowany w języku C/C++ przy wykorzystaniu nowoczesnego łańcucha narzędzi.

Architektura systemu łączy w sobie rygorystyczne standardy inżynierii oprogramowania (struktura, system budowania) z determinizmem wymagany przez systemy sterowania (RTOS). Poniżej przedstawiono szczegóły dotyczące organizacji projektu oraz logiki działania systemu czasu rzeczywistego.

2.1 Organizacja projektu i środowisko programistyczne

W celu zapewnienia czytelności, łatwej konserwacji oraz skalowalności oprogramowania, projekt odchodzi od płaskiej struktury plików generowanej domyślnie przez środowiska IDE na rzecz hierarchicznej architektury modułowej.

2.1.1 Struktura plików i katalogów

Kod źródłowy został podzielony na dwie główne domeny: logikę aplikacji (Core) oraz sterowniki komponentów zewnętrznych (Components). Drzewo projektu zorganizowano w następujący sposób:

- **Katalog Core/:** Zawiera pliki związane z konfiguracją mikrokontrolera oraz główną pętlą systemu operacyjnego.
 - **Src/Task/** oraz **Inc/Task/:** W tych podkatalogach umieszczono implementację trzech głównych wątków systemu: **PID_Task**, **Interface_Task** oraz **Communication_Task**. Taka separacja pozwala na niezależną edycję logiki regulacji i interfejsu użytkownika bez ryzyka naruszenia innych modułów.
 - **freertos.c:** Plik konfiguracyjny systemu czasu rzeczywistego, definiujący uchwyt zadań, semaforey (w tym semafor timera) oraz muteksy.
 - **data.cpp/.hpp:** Pliki definiujące globalne struktury danych (**SystemData**, **SystemConfig**) współdzielone pomiędzy wątkami.
- **Katalog Components/:** Zawiera biblioteki obsługujące zewnętrzne układy peryferyjne. Moduły te są niezależne od głównej logiki programu (Hardware Abstraction).
 - **bmp280/:** Sterownik cyfrowego czujnika temperatury i ciśnienia (I2C).
 - **Encoder/:** Biblioteka obsługująca enkoder obrotowy (filtracja drgań, logika zliczania).
 - **LCD_Library/:** Sterownik wyświetlacza alfanumerycznego dla ekspandera PCF8574.
- **Katalog cmake/:** Zawiera skrypty konfiguracyjne systemu budowania.

2.1.2 System budowania (CMake) i Dokumentacja (Doxygen)

W projekcie wykorzystano nowoczesny system budowania **CMake**, co zapewnia niezależność od środowiska IDE, przejrzystość zależności oraz automatyzację procesu kompilacji. Dodatkowo, w celu zachowania wysokiej jakości kodu, przyjęto standard dokumentowania funkcji zgodny z formatem **Doxygen**. Każdy plik nagłówkowy oraz kluczowe funkcje opatrzone są sformalizowanymi blokami komentarzy (`@brief`, `@param`, `@retval`), co pozwala na automatyczne generowanie dokumentacji technicznej.

2.2 System operacyjny czasu rzeczywistego (FreeRTOS)

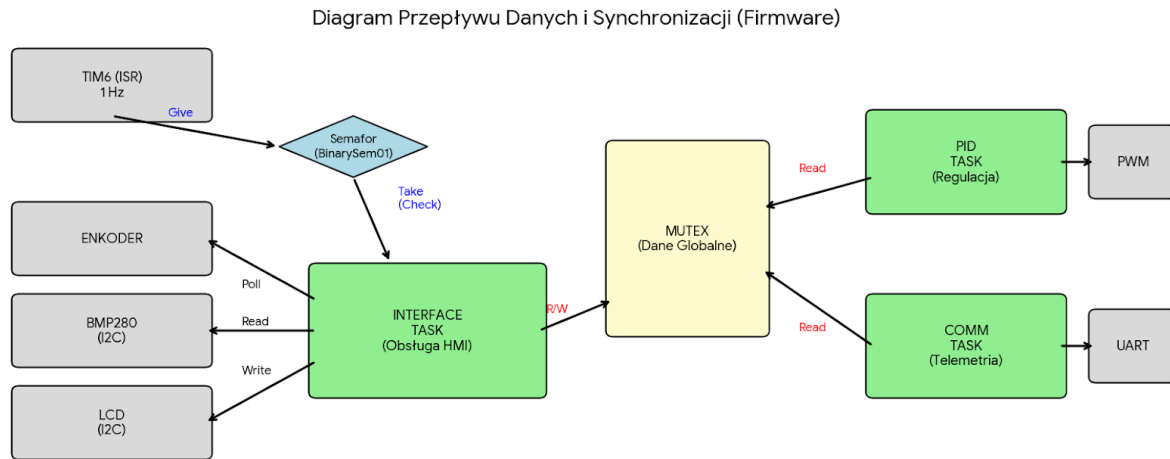
W tradycyjnych systemach wbudowanych często stosuje się architekturę "Super Loop". Ze względu na konieczność równoległej obsługi krytycznego czasowo regulatora PID oraz wolnych peryferiów (LCD, UART), w projekcie zastosowano system operacyjny z wywłaszczaniem (*Preemptive Scheduling*) – FreeRTOS.

Jądro systemu (Kernel) odpowiada za:

- **Szeregowanie zadań (Scheduling):** Planista decyduje, który wątek ma dostęp do procesora, bazując na priorytetach. Zadania o wyższym priorytecie (Regulator) wywłaszczają zadania o niższym priorytecie (Interfejs).
- **Zarządzanie czasem:** Obsługa opóźnień systemowych bez blokowania pracy procesora.
- **Komunikację międzyprocesową:** Bezpieczna wymiana danych pomiędzy wątkami.

2.3 Diagram przepływu danych i synchronizacji

Aby zilustrować zależności między zadaniami (Tasks), przerwaniami sprzętowymi oraz mechanizmami synchronizacji (Mutex, Semaphore), przygotowano graf przepływu danych przedstawiony na Rysunku 2.



Rysunek 2: Diagram przepływu danych i synchronizacji w systemie FreeRTOS

Graf ten obrazuje kluczową koncepcję projektową: wykorzystanie timera sprzętowego do cyklicznego wyzwalania odczytów czujnika (poprzez semafor), co odciąża procesor, oraz centralną rolę Mutexa w zapewnieniu spójności danych wymienianych między niezależnymi wątkami.

2.4 Szczegółowy opis zadań systemowych (Tasks)

Aplikacja została zdekomponowana na trzy autonomiczne zadania, z których każde posiada własny kontekst i stos pamięci.

2.5 Model danych i współdzielenie zasobów

Centralnym punktem wymiany informacji w systemie są globalne struktury danych zdefiniowane w pliku `data.hpp`. Zgrupowano je w logiczne bloki, co ułatwia zarządzanie stanem urządzenia.

```

1 typedef struct
2 {
3     float measured_value;    // Aktualna temperatura (PV)
4     float control_signal;    // Sygnal sterujący (CV)
5     float pid_error;        // Uchyb regulacji
6
7     // Zmienne wewnętrzne regulatora (pamięć czlonu całkującego)
8     float pid_integrator;
9
10    // Flagi stanu wyjść cyfrowych
11    bool is_fan_on;
12    bool is_ready_on;
13 } SystemData_t;
14
15 typedef struct

```

```

16 {
17     float reference_value; // Temperatura zadana (SP)
18     // Nastawy PID (modyfikowalne przez UART)
19     float kp;
20     float ki;
21     float kd;
22 } SystemConfig_t;

```

Listing 1: Struktury danych globalnych systemu

Dostęp do tych struktur z poziomu różnych wątków jest zabezpieczony mechanizmem **Mutex** (*Mutual Exclusion*). Każda operacja odczytu lub zapisu jest atomowa z punktu widzenia logiki aplikacji, co eliminuje ryzyko niespójności danych (tzw. *Race Condition*).

2.5.1 Zadanie sterowania (PID_Task)

Zadanie o **najwyższym priorytecie**, realizujące dyskretny algorytm regulacji. Kluczowe funkcjonalności zaimplementowane w tym zadaniu to:

1. **Obliczenia PID:** Dyskretna realizacja algorytmu z zabezpieczeniem przed nasyceniem członu całkującego (*Anti-Windup*).
2. **Histereza wentylatora:** Logika dwustawna załączająca chłodzenie po przekroczeniu progu COOLING_ON_DIFF.
3. **Blokada (Interlock):** Programowe zabezpieczenie uniemożliwiające jednoczesną pracę grzałki i wentylatora.
4. **Logika Okienkowa LED:** Sygnalizacja osiągnięcia temperatury zadanej z przykładową tolerancją $\pm 5\%$.

```

1 // Logika Histerezy Wentylatora
2 if (diff_cooling >= COOLING_ON_DIFF) {
3     fan_status = true; // Wlacz chlodzenie
4 }
5 else if (diff_cooling <= COOLING_OFF_DIFF) {
6     fan_status = false; // Wylacz chlodzenie
7 }
8
9 // Obliczenie sygnalu sterujacego
10 pid_out = PID_Calculate(temp_ref, temp_meas, dt_sec);
11
12 // BLOKADA (Safety Interlock)
13 // Jesli wentylator jest aktywny, wymus wylaczenie grzalki (PWM = 0)
14 if (fan_status == true) {
15     pid_out = 0.0f;
16 }

```

Listing 2: Fragment logiki sterowania w PID_Task.c

2.5.2 Zadanie interfejsu użytkownika (Interface_Task)

Odpowiada za obsługę lokalnego HMI (Enkoder + LCD) oraz akwizycję danych z czujnika BMP280. Ciekawym elementem architektury tego wątku jest synchronizacja z timerem sprzętowym. Zamiast programowego opóźnienia, odczyt czujnika jest wyzwalany przez semafor BinarySem01, który jest "dawany"(*give*) w przerwaniu timera co 1 sekundę.

```
1 // Oczekiwanie na semafor od Timera (Timeout = 0, nie blokuje enkodera)
2 if (osSemaphoreAcquire(BinarySem01Handle, 0) == osOK)
3 {
4     // Wykonuje sie tylko raz na sekunde
5     float temp_read = BMP280_ReadTemperature(&bmp280);
6
7     // Aktualizacja zmiennej globalnej (pod ochrona Mutexa)
8     if (osMutexAcquire(DataMHandle, 10) == osOK)
9     {
10         g_system_data.measured_value = temp_read;
11         osMutexRelease(DataMHandle);
12     }
13 }
```

Listing 3: Synchronizacja odczytu czujnika semaforem

Taka konstrukcja sprawia, że interfejs (enkoder) pozostaje responsywny (działa w szybkiej pętli `while`), a wolna operacja odczytu I2C jest wykonywana rzadko i deterministycznie.

2.5.3 Zadanie komunikacji (Communication_Task)

Moduł ten odpowiada za dwukierunkową wymianę danych z komputerem PC. Zastosowano tu hybrydowe podejście do obsługi interfejsu UART, łączące wydajność przerwań z prostotą cyklicznego wysyłania danych.

2.5.4 Odbiór komend (Interrupt-Based RX)

Aby uniknąć ciągłego odpytywania układu UART (polling), odbiór danych realizowany jest w trybie przerwaniowym. Funkcja zwrotna (*Callback*) jest wywoływana automatycznie przez sprzęt po odebraniu każdego bajtu. Kompletna ramka danych (zakończona znakiem nowej linii) jest sygnalizowana flagą `cmd_received`, którą następnie przetwarza wątek główny.

```
1 void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart)
2 {
3     if (huart->Instance == USART2)
4     {
5         // Sprawdzenie znaku konca linii
6         if (rx_byte == '\n' || rx_byte == '\r')
7         {
8             rx_buffer[rx_index] = 0; // Dodanie null-terminatora
9             cmd_received = 1;         // Ustawienie flagi dla watku
10            rx_index = 0;
11        }
12        else
13        {
```

```

14         // Buforowanie danych
15         rx_buffer[rx_index++] = rx_byte;
16     }
17     // Ponowne uzbrojenie przerwania
18     HAL_UART_Receive_IT(&huart2, (uint8_t*)&rx_byte, 1);
19 }
20 }

```

Listing 4: Obsługa przerwania odbiorczego UART

2.5.5 Nadawanie telemetry (Cyclic TX)

Wątek cyklicznie (co 500 ms) pobiera stan systemu, formatuje go do postaci ramki JSON i wysyła przez UART. Wykorzystanie formatu JSON ułatwia parsowanie danych w aplikacji PC, a dodanie sumy kontrolnej CRC-8 zapewnia integralność transmisji.

```

1 int len = snprintf(tx_buffer, sizeof(tx_buffer),
2     "{\"sp\":%.2f,\"pv\":%.2f,\"pwm\":%.1f,\"kp\":%.2f,\"ki\":%.3f,\"kd\":%.2f
3     ,\"fan\":%d,\"rdy\":%d}\",
    sp, pv, pwm_pct, k_p, k_i, k_d, fan_state, rdy_state);

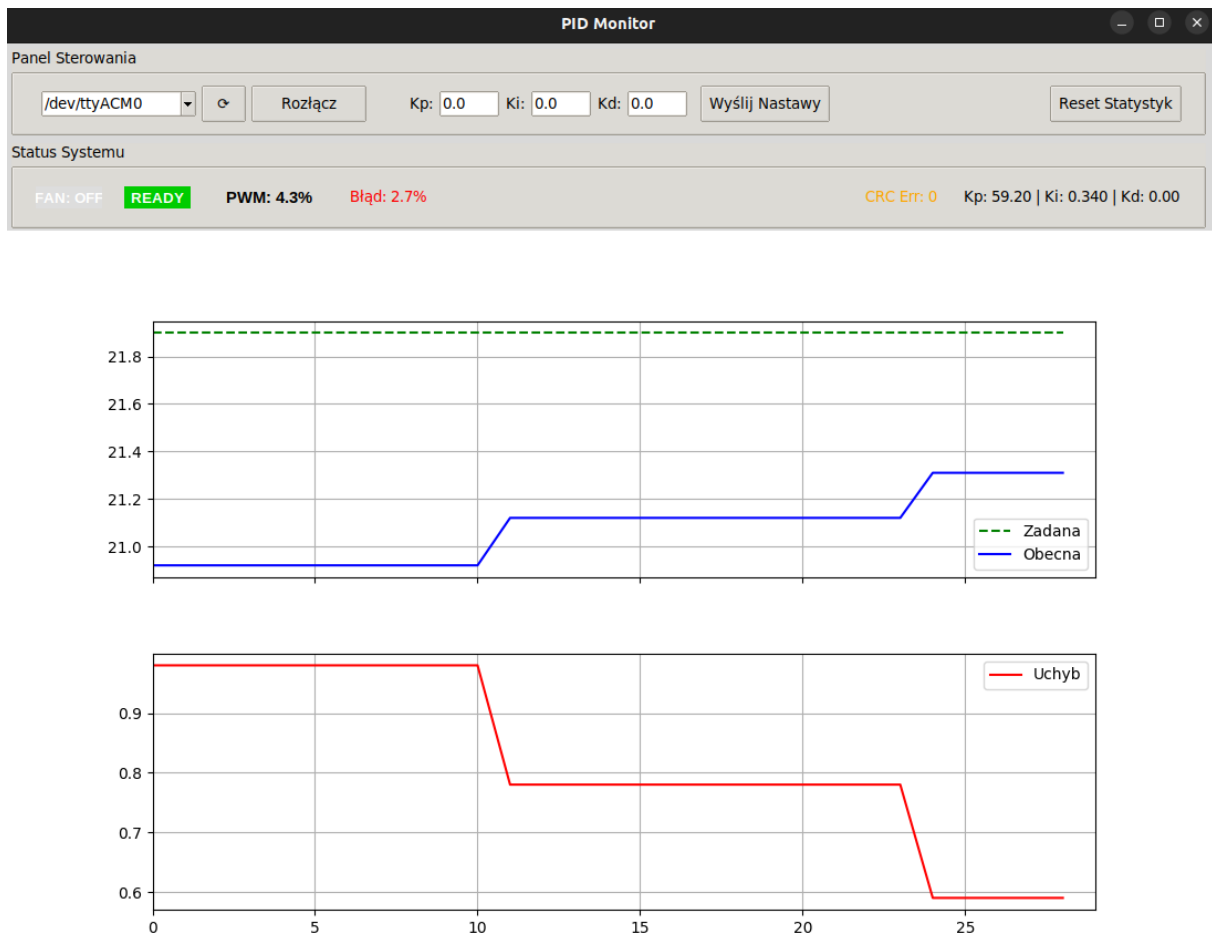
```

Listing 5: Formatowanie ramki telemetrycznej JSON

3 Aplikacja monitorująca i wizualizacyjna (PC)

Integralną częścią systemu jest dedykowana aplikacja komputerowa, pełniąca rolę zaawansowanego interfejsu HMI (*Human-Machine Interface*). Umożliwia ona wizualizację przebiegu procesu regulacji w czasie rzeczywistym, diagnostykę stanów logicznych sterownika oraz zdalne strojenie regulatora PID.

Aplikacja została zaimplementowana w języku **Python** z wykorzystaniem biblioteki **Tkinter** do budowy interfejsu graficznego oraz **Matplotlib** do generowania wykresów. Komunikacja z mikrokontrolerem realizowana jest poprzez wirtualny port szeregowy (USB CDC) przy użyciu biblioteki **PySerial**.



Rysunek 3: Zrzut ekranu aplikacji monitorującej podczas pracy układu regulacji

3.1 Architektura wielowątkowa aplikacji

Aby zapewnić płynność działania interfejsu graficznego przy jednoczesnym ciągłym odbieraniu danych z wysoką częstotliwością, aplikacja działa w oparciu o architekturę wielowątkową:

- **Wątek Główny (GUI Thread):** Odpowiada za renderowanie okna, obsługę zdarzeń (kliknięcia przycisków) oraz odświeżanie wykresów. Wykorzystuje mechanizm `root.after()` do cyklicznego pobierania danych z buforów i aktualizacji etykiet co 100 ms.

- **Wątek Komunikacyjny (Background Thread):** Działa w tle (*Daemon*), realizując ciągły nasłuch na porcie COM. Jego zadaniem jest odczyt surowych bajtów, weryfikacja sumy kontrolnej oraz parsowanie danych do struktur obiektowych.

3.2 Protokół komunikacyjny i parsowanie danych

Warstwa komunikacyjna aplikacji implementuje ten sam protokół, co oprogramowanie układowe mikrokontrolera. Każda odebrana linia tekstu jest traktowana jako potencjalna ramka danych składająca się z części JSON oraz sumy kontrolnej CRC-8.

Proces przetwarzania ramki w wątku odbiorczym przebiega następująco:

1. **Separacja:** Rozdzielenie ciągu JSON od sumy CRC (separator '|').
2. **Weryfikacja:** Obliczenie CRC-8 dla części JSON i porównanie z otrzymaną wartością HEX. Jeśli sumy się nie zgadzają, ramka jest odrzucana (inkrementacja licznika `crc_errors`).
3. **Deserializacja:** Konwersja tekstu JSON na słownik Python i aktualizacja zmiennych stanu (PWM, Fan, Ready, PID).

```

1 def read_loop(self):
2     while self.is_reading:
3         try:
4             line = self.ser.readline().decode('utf-8').strip()
5             parts = line.split('|')
6
7             # Weryfikacja integralności danych
8             json_str, crc_hex = parts[0], parts[1]
9             if self.calc_crc8(json_str) != int(crc_hex, 16):
10                 self.crc_errors += 1
11                 continue
12
13             # Parsowanie danych telemetrycznych
14             d = json.loads(json_str)
15             self.live_pwm = float(d.get("pwm", 0))
16             self.fan_status = int(d.get("fan", 0))
17             # ... (aktualizacja buforów wykresów)
18         except: pass

```

Listing 6: Fragment wątku odbiorczego w Pythonie

3.3 Funkcjonalności interfejsu

Interfejs graficzny, przedstawiony na Rysunku 3, został podzielony na trzy sekcje funkcjonalne:

1. **Panel Sterowania:** Umożliwia wybór portu szeregowego oraz nawiązanie połączenia. Kluczowym elementem jest sekcja nastaw PID, która pozwala użytkownikowi wprowadzić nowe współczynniki K_p , K_i , K_d i wysłać je do sterownika. Komenda wysyłana jest w formacie tekstowym `PID:Kp:Ki:Kd` n, który jest natychmiast interpretowany przez przerwanica w STM32.

2. **Pasek Statusu:** Prezentuje kluczowe wskaźniki w formie wizualnej:

- **FAN:** Wskaźnik binarny zmieniający kolor na niebieski, gdy wentylator jest aktywny.
- **READY:** Wskaźnik zmieniający kolor na zielony, gdy temperatura mieści się w tolerancji $\pm 5\%$ wartości zadanej.
- **Dane liczbowe:** Aktualne wypełnienie PWM, błąd względny regulacji oraz licznik błędów transmisji CRC.

3. **Wizualizacja (Wykresy):** Dwa dynamiczne wykresy rysowane przy użyciu `Matplotlib`. Górny wykres przedstawia przebieg wartości zadanej (linia przerywana) oraz mierzonej temperatury. Dolny wykres obrazuje uchyb regulacji w czasie, co pozwala na szybką ocenę jakości strojenia regulatora (przeregulowania, czas ustalania).

4 Realizacja sprzętowa urządzenia

Projekt urządzenia został zrealizowany w oparciu o modułową budowę sprzętową. Schemat elektryczny układu podzielono na logiczne sekcje funkcjonalne: sekcję mikrokontrolera (MCU), sekcję interfejsu użytkownika, sekcję pomiarową oraz stopień mocy (grzanie i chłodzenie). Poniżej przedstawiono szczegółowy opis poszczególnych bloków.

4.1 Jednostka centralna (MCU Section)

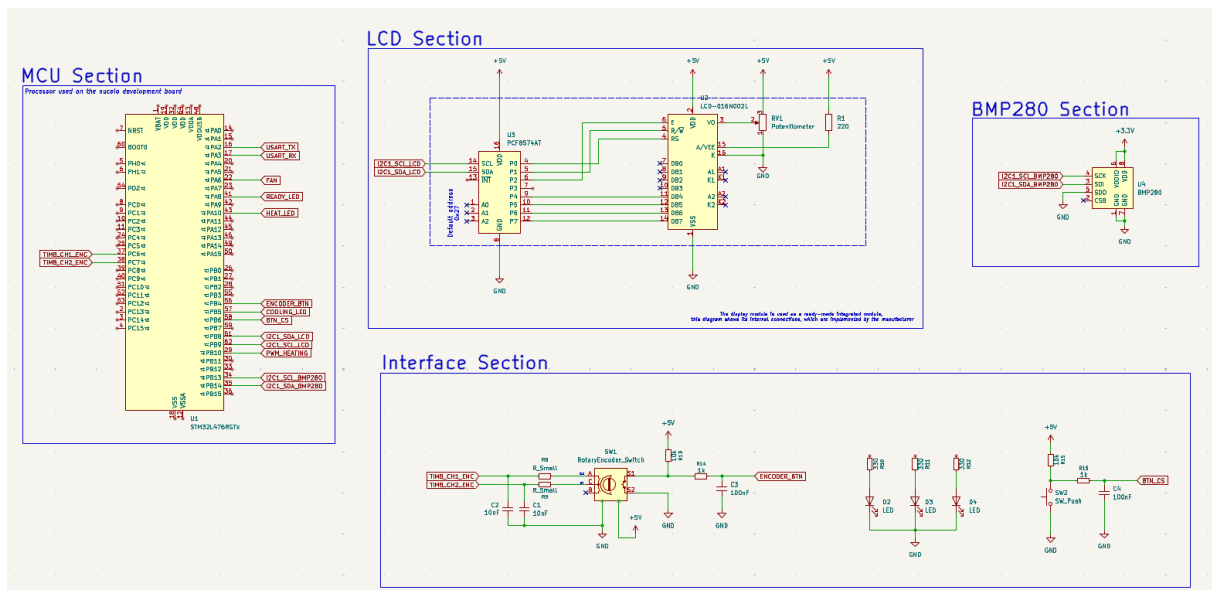
Sercem układu jest mikrokontroler z rodziny STM32 (model STM32L476RGTx), osadzony na płytce rozwojowej Nucleo. Jednostka ta odpowiada za sterowanie wszystkimi peryferiami oraz realizację algorytmu regulacji. Kluczowe połączenia pinów mikrokontrolera obejmują:

- **Magistrala I2C** : Wykorzystywana do komunikacji z czujnikiem BMP280 oraz wyświetlaczem LCD.
- **Timer enkodera (PC6, PC7)**: Piny skonfigurowane jako wejścia licznika sprzętowego (TIM8_CH1/CH2) do obsługi impulsatora obrotowego.
- **Wyjście PWM (PB10)**: Generuje sygnał sterujący o zmiennym wypełnieniu dla układu grzejnego.
- **Sterowanie wentylatorem (PA6)**: Cyfrowe wyjście GPIO sterujące pracą wentylatora.
- **Interfejs UART (PA2, PA3)**: Zapewnia komunikację szeregową z komputerem PC.

Dodatkowo, układ wyposażono w diody sygnalizacyjne LED podłączone do portów wyjściowych przez rezystory ograniczające prąd, informujące o stanie gotowości (READY_LED), grzaniu (HEAT_LED) oraz chłodzeniu (COOLING_LED).

4.2 Tor pomiarowy i interfejs użytkownika (Sensor & HMI)

Interfejs użytkownika oraz akwizycja danych zostały zrealizowane przy użyciu standardowych protokołów komunikacyjnych oraz filtracji sprzętowej.



Rysunek 4: Schemat ideowy: Mikrokontroler, czujnik BMP280 oraz interfejs LCD i enkodera

4.2.1 Sekcja wyświetlacza (LCD Section)

W projekcie zastosowano wyświetlacz alfanumeryczny LCD 2x16. Aby zminimalizować liczbę zajętych pinów mikrokontrolera, sterowanie wyświetlaczem odbywa się poprzez 8-bitowy ekspander wejść/wyjść **PCF8574AT**. Układ ten komunikuje się z MCU poprzez magistralę I2C, konwertując dane szeregowe na sygnały równoległe sterujące sterownikiem HD44780 wyświetlacza. Potencjometr RV1 służy do sprzętowej regulacji kontrastu ekranu.

4.2.2 Sekcja enkodera (Interface Section)

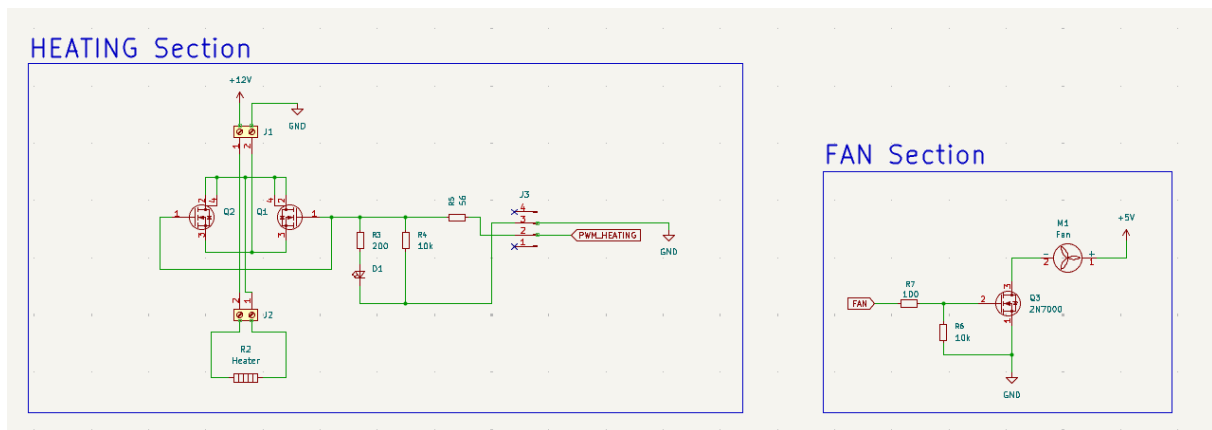
Zadawanie temperatury odbywa się za pomocą mechanicznego enkodera obrotowego z przyciskiem (SW1). Aby wyeliminować drgania styków (bouncing), typowe dla elementów mechanicznych, zastosowano sprzętowe filtry dolnoprzepustowe RC (rezystory i kondensatory C1, C2 o wartości 10nF) na liniach sygnałowych A i B. Przycisk enkodera (SW_Push) jest również podciągnięty do zasilania (Pull-up) i filtrowany kondensatorem C3.

4.2.3 Sekcja czujnika (BMP280 Section)

Elementem sprzężenia zwrotnego jest zintegrowany czujnik ciśnienia i temperatury BMP280 (U4). Układ zasilany jest napięciem 3.3V i pracuje na wspólnej magistrali I2C1. Adres urządzenia na magistrali jest konfigurowany sprzętowo poprzez stan pinu SDO.

4.3 Elementy wykonawcze (Actuators)

Część wykonawcza odpowiada za fizyczną zmianę temperatury obiektu. Składa się z dwóch niezależnych obwodów sterowanych przez mikrokontroler.



Rysunek 5: Schemat ideowy: Stopień mocy elementu grzejnego i sterowanie wentylatorem

4.3.1 Sekcja grzewcza (Heating Section)

Jako element grzejny wykorzystano rezystor mocy (R2 Heater). Sterowanie mocą odbywa się poprzez tranzystory polowe MOSFET w konfiguracji klucza. Sygnał PWM_HEATING z mikrokontrolera steruje bramką tranzystora, co pozwala na cykliczne załączanie i wyłączanie prądu płynącego przez rezystor. Dzięki dużej częstotliwości PWM oraz inercji cieplnej rezystora, uzyskuje się efekt płynnej regulacji mocy średniej. Układ wyposażono w diodę sygnalizacyjną D1 wskazującą aktywność grzania.

4.3.2 Sekcja chłodzenia (Fan Section)

Jako element wymuszający przepływ powietrza (zakłócenie) zastosowano wentylator DC (M1 Fan) zasilany napięciem 5V. Sterowanie realizowane jest w układzie "Low-Side Switch" przy użyciu tranzystora N-MOSFET typu 2N700 (Q3). Sygnał sterujący FAN podawany jest na bramkę tranzystora przez rezystor R7 (100Ω), a rezystor R8 (10kΩ) pełni funkcję pull-down, zapewniając pewne wyłączenie tranzystora w przypadku braku sygnału sterującego.

5 Identyfikacja parametryczna obiektu i synteza układu regulacji

Projektowanie efektywnego układu regulacji automatycznej wymaga znajomości dynamiki sterowanego procesu. W przypadku obiektów cieplnych, charakteryzujących się dużą bezwładnością oraz występowaniem opóźnień, dobór nastaw metodą prób i błędów jest nieefektywny i może prowadzić do niestabilności. Dlatego w niniejszym projekcie zastosowano podejście oparte na modelu (*Model-Based Design*). Proces ten podzielono na trzy etapy: eksperyment identyfikacyjny w pętli otwartej, estymację parametrów modelu matematycznego oraz analityczne strojenie regulatora PID.

5.1 Metodyka eksperymentu czynnego

Pierwszym krokiem było przeprowadzenie eksperymentu czynnego w pętli otwartej (*Open Loop*). Procedura ta polegała na wymuszeniu skokowym sygnału sterującego i rejestracji odpowiedzi układu (temperatury) w dziedzinie czasu.

Eksperyment przeprowadzono w następujących warunkach:

- **Punkt pracy:** Układ został wstępnie ustabilizowany w temperaturze otoczenia.
- **Sygnał wymuszający:** Zastosowano skok jednostkowy wypełnienia sygnału PWM z wartości 0% na 50%. Wybór wartości 50% (a nie 100%) podyktowany był chęcią uniknięcia nasycenia sensora oraz zachowania liniowości modelu w typowym zakresie pracy urządzenia.
- **Rejestracja danych:** Pomiary temperatury były próbkowane z częstotliwością 1 Hz i przesyłane drogą telemetryczną do środowiska MATLAB.

Odpowiedź układu (widoczna na Rysunku 6) wykazuje cechy charakterystyczne dla obiektów cieplnych:

1. Brak natychmiastowej reakcji na wymuszenie (występowanie czasu martwego).
2. Aperiodyczny, wykładniczy wzrost temperatury dążący do nowego stanu ustalonego.
3. Brak oscylacji, co potwierdza charakter inercyjny obiektu.

5.2 Wybór struktury modelu i estymacja parametrów

Na podstawie analizy kształtu charakterystyki skokowej, do opisu dynamiki obiektu przyjęto model inercyjny pierwszego rzędu z opóźnieniem transportowym (FOPDT – *First Order Plus Dead Time*). Jest to najczęściej stosowany model w automatyce procesowej, opisany transmitancją operatorową:

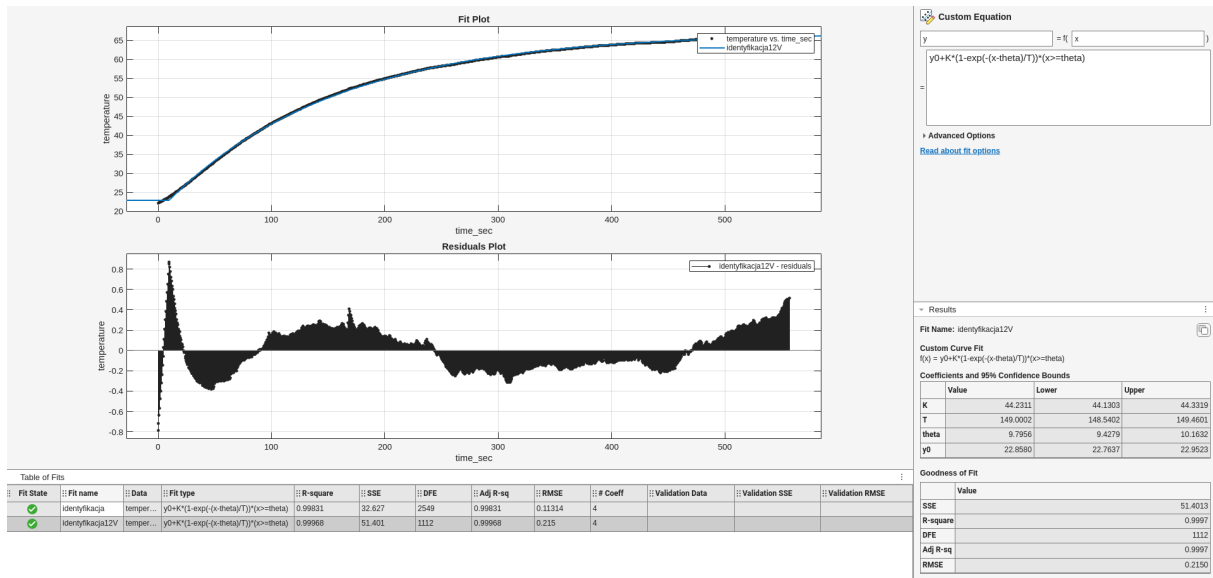
$$G(s) = \frac{Y(s)}{U(s)} = \frac{K}{Ts + 1} e^{-\theta s} \quad (1)$$

Gdzie parametry modelu mają następującą interpretację fizyczną:

- **Wzmocnienie statyczne (K):** Określa, o ile stopni Celsjusza wzrośnie temperatura przy zmianie sterowania o 1%. Jest miarą efektywności grzałki i izolacji termicznej układu.

- **Stała czasowa (T):** Definiuje bezwładność cieplną układu (czas, po którym odpowiedź osiąga 63.2% wartości ustalonej). Zależy od masy elementu grzejnego i jego ciepła właściwego.
- **Opóźnienie transportowe (θ):** Czas upływający od zmianyysterowania do momentu rozpoczęcia wzrostu temperatury. Wynika z odległości fizycznej czujnika od grzałki oraz czasu propagacji ciepła w materiale.

Do wyznaczenia wartości liczbowych parametrów wykorzystano narzędzie *Curve Fitting Tool* środowiska MATLAB, stosując metodę najmniejszych kwadratów (algorytm Levenberga-Marquardta).



Rysunek 6: Proces dopasowywania modelu FOPDT do danych pomiarowych w środowisku MATLAB (niebieska linia - model, czarne punkty - pomiary)

W wyniku estymacji uzyskano następujące parametry modelu:

Tabela 1: Wyniki identyfikacji parametrycznej obiektu

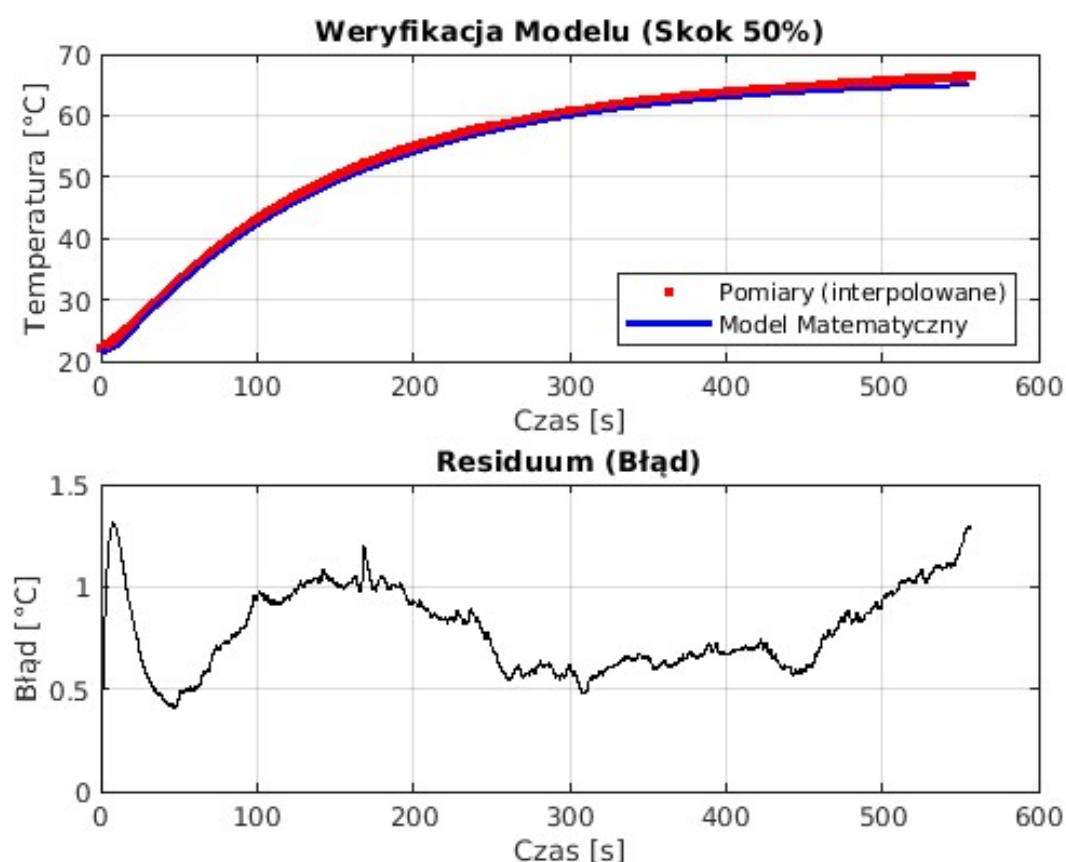
Parametr	Symbol	Wartość
Wzmocnienie statyczne	K	0.8846 [°C/%]
Stała czasowa	T	149.00 s
Opóźnienie transportowe	θ	9.80 s

Stosunek stałej czasowej do opóźnienia $\tau = \frac{T}{\theta} \approx 15.2$ wskazuje, że obiekt jest dominująco inercyjny (opóźnienie jest małe w stosunku do inercji), co jest korzystne z punktu widzenia sterowania i pozwala na zastosowanie regulatorów o dużej dynamice.

5.3 Weryfikacja jakości modelu i analiza błędów

W celu oceny przydatności wyznaczonego modelu do syntezy regulatora, przeprowadzono analizę statystyczną jakości dopasowania.

Na Rysunku 7 przedstawiono wykres porównawczy (górny) oraz wykres residuów (dolny). Residuum zdefiniowane jest jako różnica między temperaturą zmierzoną a obliczoną przez model w danej chwili czasu: $r(t) = y_{pom}(t) - y_{mod}(t)$.



Rysunek 7: Weryfikacja modelu: Porównanie odpowiedzi czasowych (góra) oraz przebieg błędu aproksymacji - residuum (dół)

Analiza wskaźników jakości dopasowania:

- **Współczynnik determinacji (R^2):** Wynosi 0.9997, co oznacza, że model wyjaśnia 99.97% zmienności danych pomiarowych.
- **Błąd średniokwadratowy (RMSE):** Wynosi 0.8141°C . Oznacza to, że model myli się średnio o niespełna 1 stopień Celsjusza.

Analizując wykres residuum (Rys. 7, dół), można zauważyć, że błąd nie jest szumem białym, lecz ma charakter deterministyczny (fluktuacje w zakresie -0.8°C do $+1.3^\circ\text{C}$). Wynika to z uproszczenia modelu (pominięcie nieliniowości wymiany ciepła oraz dynamiki czujnika). Mimo to, tak wysoka dokładność dopasowania jest w pełni wystarczająca do zaprojektowania stabilnego układu regulacji.

5.4 Synteza regulatora PID

W oparciu o zidentyfikowaną transmitancję $G(s)$, przeprowadzono strojenie regulatora PID. Celem było uzyskanie kompromisu pomiędzy krótkim czasem narastania (*Rise Time*) a minimalnym

przeregulowaniem (*Overshoot*).

Wykorzystano metody częstotliwościowe dostępne w pakiecie *Simulink Control Design*, co pozwoliło na dobranie nastaw zapewniających odpowiedni zapas fazy i modułu.

Finalne nastawy zaimplementowane w mikrokontrolerze (widoczne w aplikacji monitorującej) to:

- **Wzmocnienie proporcjonalne (K_p):** 59.20
- **Wzmocnienie całkujące (K_i):** 0.340
- **Wzmocnienie różniczkujące (K_d):** 0.00

Człon różniczkujący (D) został celowo pominięty ($K_d = 0$), ponieważ w obiektach cieplnych o dużej inercji często wzmacnia on jedynie szумы pomiarowe, nie wnosząc istotnej poprawy jakości regulacji. Zastosowanie tak dużego wzmocnienia K_p jest możliwe dzięki dużej stałej czasowej obiektu ($T = 149s$), która naturalnie tłumi gwałtowne zmiany sygnału sterującego.

5.5 Analityczny dobór nastaw metodą lokowania biegunów

Jako alternatywę dla strojenia numerycznego, przeprowadzono analityczny dobór nastaw regulatora PI metodą lokowania biegunów (ang. *Pole Placement*), zwaną w inżynierii procesowej metodą strojenia Lambda (*Lambda Tuning*).

Metoda ta polega na takim doborze parametrów regulatora, aby skompensować dominującą stałą czasową obiektu, a dynamikę układu zamkniętego ukształtować jako obiekt inercyjny I rzędu o zadanym czasie reakcji λ (stała czasowa układu zamkniętego).

5.5.1 Wyprowadzenie zależności

Przyjęto transmitancję obiektu wyznaczoną w procesie identyfikacji:

$$G(s) = \frac{K}{Ts + 1} e^{-\theta s} \quad (2)$$

Zastosowano regulator PI w strukturze standardowej (z czasem zdwojenia):

$$G_R(s) = K_p \left(1 + \frac{1}{T_i s} \right) = K_p \frac{T_i s + 1}{T_i s} \quad (3)$$

Podstawowym założeniem metody jest kompensacja bieguna obiektu przez zero regulatora. W tym celu przyjmujemy czas zdwojenia równy stałej czasowej obiektu:

$$T_i = T \quad (4)$$

Transmitancja pętli otwartej (dla zakresu częstotliwości, w którym pomijamy opóźnienie) przyjmuje postać:

$$L(s) = G_R(s)G(s) = \left(K_p \frac{T_i s + 1}{T_i s} \right) \left(\frac{K}{Ts + 1} \right) = \frac{KK_p}{Ts} \quad (5)$$

Zakładając, że oczekiwana transmitancja układu zamkniętego $G_{cl}(s)$ ma mieć postać inercyjną o stałej czasowej λ :

$$G_{cl}(s) = \frac{L(s)}{1 + L(s)} = \frac{1}{\lambda s + 1} \quad (6)$$

Po podstawieniu i przekształceniu otrzymujemy wzór na wzmocnienie regulatora:

$$\frac{KK_p}{Ts + KK_p} = \frac{1}{\frac{T}{KK_p}s + 1} \implies \lambda = \frac{T}{KK_p} \quad (7)$$

Stąd ostateczne wzory na nastawy regulatora:

$$K_p = \frac{T}{K\lambda}, \quad T_i = T \quad (8)$$

5.5.2 Obliczenia dla zidentyfikowanego modelu

Parametry obiektu wyznaczone w sekcji 4.1:

- $K = 0.8846 [^{\circ}C/\%]$
- $T = 149.00 [s]$
- $\theta = 9.80 [s]$

Jako parametr projektowy λ (pożądana stała czasowa układu zamkniętego) przyjęto wartość bezpieczną, zapewniającą stabilność i brak przeregulowań (odpowiedź asejsmiczna). Zgodnie z literaturą, dla układów z opóźnieniem przyjmuje się $\lambda \approx 3\theta$:

$$\lambda = 3 \cdot \theta = 3 \cdot 9.80 = 29.40 [s] \quad (9)$$

Obliczenie wzmocnienia proporcjonalnego K_p :

$$K_p = \frac{149.00}{0.8846 \cdot 29.40} = \frac{149.00}{26.007} \approx 5.73 \quad (10)$$

Obliczenie czasu zdwojenia T_i oraz przeliczenie na wzmocnienie całkujące K_i (dla struktury równoległej używanej w mikrokontrolerze $G_R(s) = K_p + \frac{K_i}{s}$):

$$T_i = T = 149.00 [s] \quad (11)$$

$$K_i = \frac{K_p}{T_i} = \frac{5.73}{149.00} \approx 0.038 \quad (12)$$

5.5.3 Wynik doboru nastaw

W wyniku analitycznej syntezy metodą lokowania biegunów, dla założonej dynamiki $\lambda = 3\theta$, otrzymano zestaw nastaw:

- $K_p = 5.73$
- $K_i = 0.038$

Wnioski: Otrzymane analitycznie nastawy są znacznie łagodniejsze (mniejsze wzmocnienie) niż te dobrane numerycznie w PID Tunerze ($K_p \approx 59$). Wynika to z przyjęcia konserwatywnego kryterium $\lambda = 3\theta$, które gwarantuje brak przeregulowań i wysoką odporność na zmiany parametrów obiektu, kosztem dłuższego czasu regulacji. W praktyce inżynierskiej takie nastawy są preferowane na start, jako bezpieczny punkt wyjścia do dalszego dostrajania.